

OG · CHEATSHEET FOR BUILDERS

The OG Builder's Cheatsheet

Storage. Chain. Compute. DA.

Everything an AI agent needs in one stack.

COAL EDITION · V1.0 · 2026

What is OG?

A modular AI Layer-1 designed for the agent economy: programmable storage, on-chain proofs, decentralized inference, and data availability — all in one stack.

Unlike general-purpose L1s, OG ships purpose-built primitives the AI workload needs out of the box. You don't compose 5 different protocols — they're already there as a coherent OS.

OG Storage

Content-addressable, replicated, queryable. Drop a JSON or PDF, get a SHA-256 root hash. Verifiable from anywhere.

OG Chain

EVM-compatible Layer-1. Anchor receipt hashes, deploy contracts, run Coal-style payment infra natively.

OG Compute

Decentralized inference marketplace. OpenAI-compatible API + TEE-backed sealed inference for sensitive prompts.

OG DA

Data availability for rollups + agent event logs. Cheap, persistent, queryable bytes.

Builder note: OG is the only L1 where Coal can run all 5 of its proof layers (Storage receipts, Chain anchoring, Compute-backed memory, KV state, DA logs) on one chain. That's why we ship here.

Network Quick Reference

Mainnet + Galileo testnet. The 30-second config you'll always come back to.

Mainnet

FIELD	VALUE
Chain ID	16601
RPC URL	https://evmrpc.0g.ai
Explorer	https://chainscan.0g.ai
Native token	0G
Storage gateway	https://storage.0g.ai
Compute broker	SDK: @0glabs/0g-serving-broker
Receipt anchor (Coal)	0x24a80A3Bb16d26D4063Ecd4B2fD64C6856E25E8b

Galileo Testnet

FIELD	VALUE
Chain ID	16601 (Galileo)
RPC URL	https://evmrpc-testnet.0g.ai
Explorer	https://chainscan-galileo.0g.ai
Faucet	https://faucet.0g.ai

Wallet config snippet (viem)

```
import { defineChain } from 'viem';

export const zeroG = defineChain({
  id: 16601,
  name: '0G Mainnet',
  nativeCurrency: { name: '0G', symbol: '0G', decimals: 18 },
  rpcUrls: { default: { http: ['https://evmrpc.0g.ai'] } },
  blockExplorers: { default: { name: '0G Scan', url: 'https://chainscan.0g.ai' } },
});
```

Watch out: The most common OG integration bug is mismatched chain IDs — Coal's `env.ts` defaults to 16601, but if your RPC URL points at testnet, every `eth_chainId` call returns 16600 and your tx attempts will silently fail. Verify with `curl -X POST $RPC -d '{"method":"eth_chainId","jsonrpc":"2.0","id":1}'` before any deploy.

OG Storage

Decentralized object store. Content-addressable. Built for verifiable JSON, blobs, and documents.

Upload a JSON document

```
import { ZeroGStorageClient } from '@0gfoundation/0g-ts-sdk';
import { Wallet } from 'ethers';

const wallet = new Wallet(process.env.OPERATOR_KEY!);
const client = new ZeroGStorageClient({
  rpcUrl: 'https://evmrpc.0g.ai',
  storageGateway: 'https://storage.0g.ai',
  signer: wallet,
});

const { rootHash, txHash } = await client.uploadJson({
  type: 'coal.receipt.v1',
  payload: { sessionId: 'cs_123', amount: '0.10', currency: 'USDC' },
});
// rootHash = '0xfb508c72dddcfb2a8448affebb37f6c083df1eb8b5b98e200a246f2838d3c1c1'
```

Retrieve by root hash

```
const raw = await fetch(`https://storage.0g.ai/file?root=${rootHash}`).then(r =>
  r.text());
const doc = JSON.parse(raw);
```

Anchor the hash on OG Chain

Storage alone gives you a content-addressable URL. To make it tamper-proof, anchor the SHA-256 of the payload on OG Chain via Coal's CoalReceiptAnchor contract:

```
await publicClient.writeContract({
  address: '0x24a80A3Bb16d26D4063Ecd4B2fD64C6856E25E8b',
  abi: ANCHOR_ABI,
  functionName: 'anchor',
  args: [rootHash, payloadSha256],
});
```

Three-step proof trail: **(1)** Storage URI → **(2)** Storage root hash → **(3)** Chain anchor. Coal calls this the *receipt trail*, and every paid checkout produces one automatically.

0G Compute

Decentralized inference marketplace. OpenAI-compatible. TEE-backed sealed inference for sensitive workloads.

Mint a session token (24h validity)

```
import { createZGComputeNetworkBroker } from '@0g Labs/0g-serving-broker';
import { Wallet, JsonRpcProvider } from 'ethers';

const wallet = new Wallet(process.env.OPERATOR_KEY!, new
JsonRpcProvider('https://evmrpc.0g.ai'));
const broker = await createZGComputeNetworkBroker(wallet);
const headers = await broker.inference.getRequestHeaders(providerAddress);
const apiKey = headers.Authorization.replace(/^Bearer /, '');
```

Call the OpenAI-compatible endpoint

```
import OpenAI from 'openai';

const openai = new OpenAI({
  apiKey,
  baseURL: 'https://integratenetwork.work/v1/proxy',
});

const chat = await openai.chat.completions.create({
  model: 'qwen3.6-plus',
  messages: [{ role: 'user', content: 'What does 0G ship?' }],
});
```

Sealed Inference (TEE)

For prompts containing PII, financial data, or regulated content, route through a sealed-inference provider. The TEE attestation proves the model never saw raw plaintext outside the enclave. Coal uses this for the `evaluate_policy` tool — merchant policies are queried without exposing the underlying rules to the inference provider.

Token TTL gotcha: 0G Compute session tokens expire every ~24h. Your server must mint fresh tokens via the broker SDK on cold-start AND retry on mid-stream "session token expired" errors. Coal's `lib/0g-token.ts` shows the cache-and-retry pattern.

Health-check pattern

Every production OG app should expose a /health endpoint that pings all 5 components. Catches RPC outages, expired tokens, and broken indexers before users do.

Ship this endpoint on day 1

```
export async function GET() {
  const checks = await Promise.allSettled([
    // Storage: round-trip a small object
    fetch(`${STORAGE_GATEWAY}/file?root=${HEALTH_HASH}`),
    // Chain: read latest block
    publicClient.getBlockNumber(),
    // Compute: confirm broker can mint a token
    broker.inference.listProviders(),
    // KV / DA: optional pings
  ]);
  const ok = checks.every(c => c.status === 'fulfilled');
  return Response.json({ status: ok ? 'ok' : 'degraded', checks });
}
```

Patterns Coal uses in production

1. Receipt trail on every payment

Each successful checkout produces: Base USDC tx → OG Storage upload (receipt JSON) → OG Chain anchor (SHA-256 of payload). Surface all three on the receipt page so buyers can independently verify.

2. Agent-discoverable catalog

Publish your product list as JSON to /llms.txt, /.well-known/agent-card.json, and /.well-known/x402.json. Coal's CoalAgentPublisher React component does all three at build time.

3. Per-call x402 paywalls

For paid APIs (oracles, scrapers, paid datasets), gate every request behind an HTTP 402 response carrying x402 payment requirements. The agent signs an EIP-3009 authorization, posts to the verify endpoint, gets the data. Coal facilitates settlement on Base — your endpoint stays purely a content server.

4. Memory queries via OG Compute

Treat 0G Compute as your memory query layer. Index your products + policies into a vector index, store the index on 0G Storage, and answer agent questions with sealed inference — the LLM provider can't see your raw data.

Resources

The links you'll bookmark on day 1 and never close.

Official OG

- **Homepage:** <https://0g.ai>
- **Docs:** <https://docs.0g.ai>
- **Explorer:** <https://chainscan.0g.ai>
- **Storage Scan:** <https://storagescan.0g.ai>
- **Faucet (testnet):** <https://faucet.0g.ai>
- **X / Twitter:** [@0G_labs](#) / [@0g_Eco](#)

Coal (this book's publisher)

- **Coal homepage:** <https://usecoal.xyz>
- **Live agent demo:** <https://agent.usecoal.xyz>
- **x402 paywall demo:** oracle.usecoal.xyz/api/price/ETH
- **MCP server:** mcp.usecoal.xyz/api/mcp
- **OG health:** api.usecoal.xyz/api/0g/health
- **SDKs:** `npm i coal-react` · `npm i create-0g-dapp`

Standards Coal speaks natively

- **x402 (Coinbase):** x402.org — HTTP 402 + signed authorization for agent payments
- **OKX APP:** github.com/okx/payments — x402 v2 envelope, byte-compatible with EIP-3009
- **EIP-3009:** eips.ethereum.org/EIPS/eip-3009 — gasless USDC transfers
- **MCP:** modelcontextprotocol.io — Anthropic's tool-use protocol for AI agents

Hackathon-ready starters

- `npx create-0g-dapp my-app` — Coal's CLI scaffolds a Next.js app with all OG primitives wired up
- `npm i coal-react` — drop-in React components for checkout, paywalls, agent-discoverable catalogs
- `/examples/coal-agent` — reference autonomous-buyer implementation
- `/examples/coal-mcp-server` — reference MCP server with 12 tools for Claude/Cursor

